

Tier 3 Specific Magic Items Implementation Plan

Overview

Tier 3 items are “Very Complex” — they require new gameplay subsystems, significant combat mechanics changes, or complex state management beyond what the existing `SpecificItemBehavior` framework provides out of the box. This document provides a complete implementation roadmap grounded in the **actual codebase** and **verified SRD descriptions**.

Source of truth: `/home/ubuntu/specific_magic_items_complete_accurate.md` (verified against SRD)

Codebase: `/home/ubuntu/dnd35prototype` (Unity C# project)

Base framework: `SpecificItemBehavior` class + combat hooks (implemented in Tier 2)

Total Tier 3 Items: 12 unique items (+ 3 Luck Blade wish-count variants = 15 entries)

Item List

#	Item	Type	Price (gp)	Core Complexity
1	Luck Blade (0/1/2/3 wishes)	Weapon	22,060–142,960	Reroll system, luck saves, wish system
2	Nine Lives Stealer	Weapon	23,057	9-charge crit- death, evil align- ment penalty
3	Oathbow	Weapon	25,600	Sworn enemy state machine, enhancement switching
4	Sword of Life Stealing	Weapon	25,715	Crit-triggered negative level + temp HP
5	Screaming Bolt	Weapon	267	AoE fear along projectile path
6	Shifter's Sorrow	Weapon	12,780	Anti- shapechanger, forced form re- vert
7	Life-Drinker	Weapon	40,320	Every-hit negat- ive levels + self- penalty
8	Sun Blade	Weapon	50,335	Triple conditional enhancement, dual proficiency, sunlight
9	Frost Brand	Weapon	54,475	Fire resistance, dispel fire ma- gic, frost
10	Dwarven Thrower	Weapon	60,312	Race-condition- al, thrown bonus damage
11	Mace of Smiting	Weapon	75,312	Conditional en- hancement, con-

#	Item	Type	Price (gp)	Core Complexity
				struct instant kill, outsider crit
12	Holy Avenger	Weapon	120,630	Class-conditional, SR aura, greater dispel
13	Celestial Armor	Armor	22,400	Light category override, extreme stats, fly
14	Demon Armor	Armor	52,260	Claw attacks, contagion, cursed, alignment penalty

Note: `PlateArmorOfEtherealness` is in our enum but is NOT in the SRD specific items list. It's included in the database for completeness but is lowest priority.

Existing Subsystems Available (UPDATED — Post-Audit May 24, 2026)

Full audit details: See `/home/ubuntu/existing_systems_audit.md` for complete API documentation, file locations, and code examples for every system below.

 **Already Implemented (can leverage directly) — 20 Systems Found**

#	System	Key File(s)	Key API	Tier 3 Items Using It
1	Haste System	CharacterCon- troller.cs: 1445-1530 , Character- Stats.cs: 1310-1313	ApplyHasteEf- fect(rounds, caster) , Haste- AttackBonus/AC- Bonus/ReflexBo- nus fields	(Tier 2 done)
2	Emanation Framework	EmanationEf- fectData.cs (abstract base), GameManager.cs: 9464	RegisterEmana- tion() , IsCreatureIn- Area() , GetCreaturesIn- Area()	Holy Avenger SR aura
3	Spell Resist- ance	Character- Stats.cs:1854	SpellResistance (int, directly set- table), SR check: 1d20 + CL vs SR in SpellCasting	Holy Avenger
4	Rage System	Character- Stats.cs: 936-1000	ActivateRage() , DeactivateR- age() , IsRaging , TickRage()	Demon Armor
5	Weapon Fin- esse	FeatManager.cs: 248-268	Should- UseWeaponFi- nesse(stats, weapon) , GetMeleeAttack- Ability- Mod(stats, weapon)	Sun Blade
6	Negative Levels	Character- Stats.cs:762	ApplyCondi- tion(CombatCond ition- Type.EnergyDrai ned, source, rounds) , Negat- iveLevelCount	Sword of Life Stealing, Life- Drinker, Nine Lives Stealer
7	Fort Save + Death	SavingThrowRe- solver.cs ,	ResolveForti- tudeSave(stats,	

#	System	Key File(s)	Key API	Tier 3 Items Using It
		Character-Stats.cs	dc, name) → set CurrentHP = -10, IsDead = true	Nine Lives Stealer, Mace of Smiting
8	Condition System	CombatConditionType.cs , Character-Stats.cs	ApplyCondition(type, source, rounds) , HasCondition() on CharacterController	Screaming Bolt, Demon Armor
9	Temporary HP	Character-Stats.cs:2358	Stats.TempHP += amount (absorbed first in TakeDamage flow)	Sword of Life Stealing
10	Energy Resistance	Character-Stats.cs:1845, 4036	SetResistEnergyEffect(ResistEnergyEffectData) with Damage-Type + Amount	Frost Brand
11	Alignment System	Alignment.cs , Character-Stats.cs:198	AlignmentHelper.IsEvil/IsGood/IsLawful/IsChaotic() , CharacterAlignment enum	Holy Avenger, Sun Blade, Nine Lives Stealer
12	Race / Class Checks	Character-Stats.cs:489, 4799	IsPaladin , HasClass("X") , GetClassLevel("X") , RaceName (string)	Dwarven Thrower, Holy Avenger
13	Dispel Magic	GameManager.DispelCounterspell.cs	PerformDispelCheck() , PerformTargetedDispel() , PerformAreaDis-	Holy Avenger

#	System	Key File(s)	Key API	Tier 3 Items Using It
			pel() , Greater = +20 cap	
14	Thrown Weapon System	ItemData.cs: 237 , CharacterController.cs: 4661	IsThrown flag, CanBeThrown , range penalty, STR on thrown damage	Dwarven Thrower
15	Creature Type System	CharacterStats.cs:1742 , SpecificItemBehavior.cs	CreatureType string, IsCreatureType() / IsCreatureTypeAny() base helpers	Mace of Smiting, Sun Blade, Shifter's Sorrow
16	Natural Attack System	CharacterStats.cs:40-80	NaturalAttackDefinition class, NaturalAttacks list	Demon Armor (claw attacks)
17	Disease System	CharacterStats.cs:64-66 , GameManager.SpellCasting.cs:2117+	HasDiseaseOnHit , DiseaseOnHitType , Contagion spell flow	Demon Armor
18	Crit Multiplier	CharacterStats.cs:1925 , CharacterController.cs:4754	CritMultiplier int, RollCritDamage()	Mace of Smiting
19	DR System	CharacterStats.cs:1803, 4314	AddDamageReduction(amount, bypassTags) , RemoveDamageReduction()	—
20	SpecificItem-Behavior Hooks	SpecificItemBehavior.cs , CharacterController.cs	OnPreAttackRoll , OnDamageRoll , OnCriticalHit , OnHitApplied , OnKill , OnAttackedBy , Ac-	All items

#	System	Key File(s)	Key API	Tier 3 Items Using It
			tivate , ApplyPassiveStatBonuses	

⚠ Partially Available (need minor extension)

System	Current State	Gap	Fix Effort
Weapon Finesse	Feat check works but requires <code>HasFeat("Weapon Finesse")</code>	Sun Blade needs built-in finesse without feat	0.25 days: add <code>GrantsFinesse</code> or behavior adds DEX-Mod-STRMod in <code>OnPreAttackRoll</code>
Crit Multiplier Override	Read from <code>Stats.CritMultiplier</code> once per attack	Mace of Smiting needs ×4 vs outsiders, ×3 vs constructs	0.25 days: override in <code>OnCriticalHit</code> with extra damage
Luck Reroll	<code>ApplyLuckReroll()</code> exists for saves (Luck Domain)	Luck Blade needs player-triggered reroll of ANY roll type	1 day: UI + state tracking
Behavior Hook Parameters	<code>OnPreAttackRoll</code> / <code>OnDamageRoll</code> don't receive <code>isRanged</code> flag	Dwarven Thrower needs to know if attack is thrown	0.25 days: add parameter or use distance heuristic

✗ Needs New Implementation — 7 Systems (down from 13 originally)

#	System	Required For	Complexity	Estimated Work
1	Sworn Enemy State Machine	Oathbow	High	1 day
2	Equipment Curse System	Demon Armor	Low	0.5 days (add IsCursed to ItemData + un-equip check)
3	Forced Shapechanger Revert	Shifter's Sorrow	Low	0.5 days (detect subtype + log note)
4	Player-Triggered Reroll	Luck Blade	Medium	1 day (UI + any-roll tracking)
5	Wish Charges	Luck Blade (1/2/3 wish)	High	0.5-2 days (simplified: charge + preset options)
6	AoE Path Targeting	Screaming Bolt	Medium	0.5 days (ray-trace on grid)
7	Fly Movement	Celestial Armor	Medium	1 day (or simplified: speed boost + "flying" tag)

Eliminated from "needs building" (already exists):

- ~~SR Aura~~ → Emanation framework exists, just subclass it
- ~~Greater Dispel Magic~~ → `PerformAreaDispel()` already exists with Greater cap (+20)
- ~~Fire Spell Dispel~~ → Can use existing dispel check flow
- ~~Claw Natural Attack~~ → `NaturalAttackDefinition` system already exists
- ~~Contagion on Hit~~ → Disease system already exists (`HasDiseaseOnHit`)
- ~~Sunlight Area Effect~~ → Can be simplified to bonus damage + log note

Tier 3 Item Detailed Breakdowns

Phase 1 — High Priority (5 items, ~6 days)

Items that primarily use existing combat hooks with moderate new logic.

1. NINE LIVES STEALER

SRD: +2 longsword. 9 charges. On critical hit vs crit-susceptible creature: DC 20 Fort save or instant death. Charge consumed only on FAILED save. Evil weapon — good-aligned wielders gain 2 negative levels (persist while wielded, can't be removed).

Enhancement: +2 longsword

Price: 23,057 gp

Implementation:

```

public class NineLivesStealerBehavior : SpecificItemBehavior
{
    private int _chargesRemaining = 9;

    public override void OnEquip(CharacterController character)
    {
        base.OnEquip(character);
        // Evil weapon: good-aligned wielders gain 2 negative levels
        if (character.Stats != null &&
            AlignmentHelper.IsGood(character.Stats.CharacterAlignment))
        {
            character.ApplyNegativeLevels(2, "Nine Lives Stealer (evil weapon)");
            Log("Good-aligned wielder gains 2 negative levels");
        }
    }

    public override void OnUnequip()
    {
        // Remove alignment penalty negative levels
        // (Would need tracking to remove exactly 2)
        base.OnUnequip();
    }

    public override void OnCriticalHit(CharacterController target, int damage,
        List<string> logNotes)
    {
        if (_chargesRemaining <= 0) return;
        if (target?.Stats == null) return;
        // No effect on crit-immune creatures (undead, constructs, etc.)
        // Check via CreatureType – constructs and oozes are crit-immune
        if (IsCreatureTypeAny(target, "Construct", "Ooze", "Plant")) return;

        var save = SavingThrowResolver.ResolveFortitudeSave(
            target.Stats, 20, "Nine Lives Stealer");

        if (!save.Succeeded)
        {
            _chargesRemaining--;
            target.Stats.CurrentHP = -100;
            target.OnDeath();

            logNotes.Add($"💀 Nine Lives Stealer claims {target.Stats.CharacterName}'s soul! "
                + $"(Fort DC 20: {save.Total}) [{_chargesRemaining}/9 charges]");
        }
        else
        {
            logNotes.Add($"Nine Lives Stealer: {target.Stats.CharacterName} resists "
                + $"(Fort DC 20: {save.Total}) [charge NOT consumed]");
        }
    }
}

```

Existing Systems Used: OnCriticalHit hook, SavingThrowResolver, ApplyNegativeLevels, AlignmentHelper.IsGood(), ItemData.CurrentCharges

New Systems Needed: Alignment-penalty negative levels that auto-remove on unequip (track count)

Estimated Time: 0.5 days

Testing:

- [] Crit triggers death save (DC 20 Fort)

- [] Charge consumed only on failed save
- [] No effect vs constructs/oozes (crit-immune)
- [] Good wielder gets 2 negative levels
- [] After 9 charges: functions as plain +2 longsword
- [] Negative levels removed on unequip

2. SWORD OF LIFE STEALING

SRD: +2 longsword. On critical hit: bestows 1 negative level. Wielder gains 1d6 temporary HP (lasts 24 hours). 24 hours later: DC 16 Fort per negative level or permanent level loss.

Enhancement: +2 longsword

Price: 25,715 gp

Implementation:

```
public class SwordOfLifeStealingBehavior : SpecificItemBehavior
{
    public override void OnCriticalHit(CharacterController target, int damage,
        List<string> logNotes)
    {
        if (target?.Stats == null || Wielder?.Stats == null) return;

        // Bestow 1 negative level
        target.ApplyNegativeLevels(1, "Sword of Life Stealing");

        // Wielder gains 1d6 temporary HP
        int tempHP = DiceService.D6("Sword of Life Stealing temp HP");
        // Note: need TemporaryHP field or AddTemporaryHP method
        Wielder.Stats.TemporaryHP += tempHP;

        logNotes.Add($"Sword of Life Stealing: {target.Stats.CharacterName} "
            + $"gains 1 negative level! {Wielder.Stats.CharacterName} gains {tempHP} "
            + "temp HP");

        // Fort DC 16 after 24 hours handled by negative level system
    }
}
```

Existing Systems Used: OnCriticalHit , ApplyNegativeLevels

New Systems Needed: CharacterStats.TemporaryHP field (if not present — needs verification)

Estimated Time: 0.5 days

Testing:

- [] Crit bestows exactly 1 negative level
- [] Wielder gains 1d6 temp HP
- [] Non-crit hits do nothing special
- [] Works with existing negative level save system

3. LIFE-DRINKER

SRD: +1 greataxe. On EVERY hit: target gains 2 negative levels. Wielder also gains 1 negative level (lasts 1 hour). Undead/construct wielders are exempt from self-penalty. DC 16 Fort 24 hours later per negative level.

Enhancement: +1 greataxe

Price: 40,320 gp

Implementation:

```
public class LifeDrinkerBehavior : SpecificItemBehavior
{
    public override void OnHitApplied(CharacterController target, int finalDamage,
        List<string> logNotes)
    {
        if (target?.Stats == null) return;

        // Target: 2 negative levels on every hit
        target.ApplyNegativeLevels(2, "Life-Drinker");
        logNotes.Add($"Life-Drinker: {target.Stats.CharacterName} gains 2 negative
levels!");

        // Wielder: 1 negative level (1 hour duration)
        // Undead/construct wielders exempt
        if (Wielder?.Stats != null)
        {
            string wielderType = Wielder.Stats.CreatureType ?? "";
            bool exempt = wielderType.Equals("Undead", Sys-
tem.StringComparison.OrdinalIgnoreCase)
                || wielderType.Equals("Construct", Sys-
tem.StringComparison.OrdinalIgnoreCase);

            if (!exempt)
            {
                Wielder.ApplyNegativeLevels(1, "Life-Drinker (self)");
                logNotes.Add($"Life-Drinker: {Wielder.Stats.CharacterName} also gains
1 negative level (1 hour)");
                // Note: 1-hour auto-removal would need a timer system
            }
        }
    }
}
```

Existing Systems Used: OnHitApplied, ApplyNegativeLevels, CreatureType

New Systems Needed: Timed negative level removal (1-hour duration vs permanent)

Estimated Time: 0.5 days

Testing:

- [] Every hit gives target 2 negative levels (not just crits)
 - [] Every hit gives wielder 1 negative level
 - [] Undead/construct wielders exempt from self-penalty
 - [] Self-penalty negative levels expire after 1 hour
-

4. MACE OF SMITING

SRD: +3 adamantine heavy mace. +5 enhancement vs constructs. Critical hit vs construct: instant destruction (NO save). Critical hit vs outsider: ×4 damage instead of ×2.

Enhancement: +3 (adamantine material)

Price: 75,312 gp

Implementation:

```

public class MaceOfSmitingBehavior : SpecificItemBehavior
{
    public override void OnPreAttackRoll(CharacterController target, ref int attackBo-
nus,
                                List<string> logNotes)
    {
        if (target?.Stats == null) return;

        // +5 vs constructs (base is +3, so +2 extra)
        if (IsCreatureType(target, "Construct"))
        {
            attackBonus += 2;
            logNotes.Add("Mace of Smiting: +5 vs construct");
        }
    }

    public override void OnDamageRoll(CharacterController target, ref int damage,
                                bool isCrit, List<string> logNotes)
    {
        if (target?.Stats == null) return;

        // +5 enhancement damage vs constructs (extra +2 over base +3)
        if (IsCreatureType(target, "Construct"))
        {
            damage += 2;
        }

        // Critical hit vs outsider: x4 instead of x2
        // This requires modifying the crit multiplier BEFORE damage calculation
        // which happens earlier in the flow. See implementation note below.
    }

    public override void OnCriticalHit(CharacterController target, int damage,
                                List<string> logNotes)
    {
        if (target?.Stats == null) return;

        // Crit vs construct: instant destruction, NO SAVE
        if (IsCreatureType(target, "Construct"))
        {
            target.Stats.CurrentHP = -100;
            target.OnDeath();
            logNotes.Add($"Mace of Smiting DESTROYS {target.Stats.CharacterName}!
(construct crit = instant destruction)");
            return;
        }

        // Crit vs outsider: x4 damage – need extra damage to simulate
        // The base crit already applied x2; we need to add another x2 worth
        if (IsCreatureType(target, "Outsider"))
        {
            // Add the weapon's base damage again to simulate x4 from x2
            // This is approximate – exact implementation needs crit multiplier over-
            ride
            logNotes.Add("Mace of Smiting: x4 critical vs outsider");
        }
    }
}

```

Existing Systems Used: OnPreAttackRoll, OnDamageRoll, OnCriticalHit, CreatureType

New Systems Needed: Crit multiplier override system (for x4 vs outsider). Could approximate by

adding extra damage in `OnCriticalHit`.

Estimated Time: 0.5 days

Implementation Note — Crit Multiplier Override:

The current crit system calculates damage based on weapon's `CritMultiplier`. To support $\times 4$ vs outsiders, options:

1. **Approximation:** In `OnCriticalHit`, calculate what the additional damage would be (base weapon damage $\times$ extra multiplier factor) and add it. Quick but slightly imprecise.
2. **Proper:** Add a `GetCritMultiplierOverride()` method to `SpecificItemBehavior` called before crit damage calculation in `CharacterController`. More work but accurate.

Recommendation: Start with approximation (option 1), refactor to option 2 if needed.

Testing:

- [] +3 vs normal creatures, +5 vs constructs (attack AND damage)
- [] Crit vs construct = instant destruction (no save)
- [] Crit vs outsider = $\times 4$ damage
- [] Normal crit vs others = standard $\times 2$
- [] Adamantine material bypasses hardness

5. SCREAMING BOLT

SRD: +2 bolt. When fired, enemies within 20 feet of the bolt's flight path must make DC 14 Will save or become shaken for 1 round. Mind-affecting fear effect. Single-use ammunition.

Enhancement: +2 bolt

Price: 267 gp

Implementation:

```

public class ScreamingBoltBehavior : SpecificItemBehavior
{
    private const int FearDC = 14;

    // The AoE fear triggers on firing, not on hit
    // In our combat flow, this fires during OnPreAttackRoll (when the bolt is
    // launched)
    public override void OnPreAttackRoll(CharacterController target, ref int attackBo-
nus,
                                     List<string> logNotes)
    {
        // Path-based AoE fear: all enemies within 20 ft of bolt path
        // Simplified: affect the target and any enemies within 20 ft of target
        // (since we don't have projectile path tracking)
        if (Wielder == null) return;

        // Find all enemies near the target (within 20 ft = 4 squares)
        logNotes.Add("Screaming Bolt: bolt screams in flight!");

        // Note: Full implementation needs access to combatant list
        // For now, apply to target only as primary effect
        // The AoE component would need GameManager.Combat_GetAllCharacters()
    }

    public override void OnHitApplied(CharacterController target, int finalDamage,
                                     List<string> logNotes)
    {
        // Apply fear to target on hit as simplified version
        if (target?.Stats == null) return;

        var save = SavingThrowResolver.ResolveWillSave(target.Stats, FearDC, "Scream-
ing Bolt");
        if (!save.Succeeded)
        {
            target.ApplyCondition(CombatConditionType.Shaken, 1, "Screaming Bolt");
            logNotes.Add($"Screaming Bolt: {target.Stats.CharacterName} is shaken!
(Will DC {FearDC}: {save.Total})");
        }
        else
        {
            logNotes.Add($"Screaming Bolt: {target.Stats.CharacterName} resists fear
(Will DC {FearDC}: {save.Total})");
        }
    }
}

```

Existing Systems Used: OnPreAttackRoll, OnHitApplied, SavingThrowResolver, ApplyCondi-
tion(Shaken)

New Systems Needed: Path-based AoE targeting (needs access to combatant list and position data
for full 20-ft radius along flight path). Can use simplified “near target” approach initially.

Estimated Time: 1 day

Simplification Strategy:

The “20-foot radius along flight path” AoE is extremely unusual and would require projectile trajectory
tracking. For the prototype:

- **Phase 1:** Apply fear effect only to the direct target (simplified).
- **Phase 2:** Find all enemies within 20 ft of the target’s position and apply fear to them too.
- **Phase 3 (optional):** Full projectile path calculation.

Testing:

- [] Target makes Will DC 14 save or shaken 1 round
- [] Mind-affecting — immune creatures unaffected
- [] Bolt is consumed after use

Phase 2 — Medium Priority (4 items, ~8 days)

Items requiring new subsystems or significant conditional logic.

6. LUCK BLADE (all variants)

SRD: +2 short sword. +1 luck bonus on ALL saving throws (passive, always). 1/day reroll any one roll just made (must accept reroll, even if worse). May contain 0-3 wishes.

Variants: 0 wishes (22,060 gp), 1 wish (62,360 gp), 2 wishes (102,660 gp), 3 wishes (142,960 gp)

Implementation:

```

public class LuckBladeBehavior : SpecificItemBehavior
{
    private const int LuckSaveBonus = 1;
    private int _wishesRemaining;
    private bool _rerollUsedToday;

    public LuckBladeBehavior(int wishes)
    {
        _wishesRemaining = wishes;
    }

    public override void ApplyPassiveStatBonuses(CharacterStats stats)
    {
        if (stats == null) return;
        // +1 luck bonus to ALL saves
        stats.MoraleSaveBonus += LuckSaveBonus;
        // Note: Should ideally be a "luck" bonus type (doesn't stack with other luck)
        // Using MoraleSaveBonus as closest available field
    }

    public override void RemovePassiveStatBonuses(CharacterStats stats)
    {
        if (stats == null) return;
        stats.MoraleSaveBonus -= LuckSaveBonus;
    }

    // Reroll: 1/day, any roll, must accept result
    public override bool CanActivate()
    {
        return IsEquipped && !_rerollUsedToday;
    }

    public override string GetActivateDescription()
    {
        string wishStr = _wishesRemaining > 0 ? $" {_wishesRemaining} wish(es) remain-
ing." : "";
        return _rerollUsedToday
            ? $"Reroll used today.{wishStr}"
            : $"Reroll any one roll just made (must accept result){wishStr}";
    }

    public override bool Activate(CharacterController target, List<string> logNotes)
    {
        if (_rerollUsedToday) return false;
        _rerollUsedToday = true;
        logNotes.Add("Luck Blade: reroll activated! (must accept new result)");
        // Actual reroll logic needs UI integration – the system must:
        // 1. Record the last roll made by the wielder
        // 2. On activation, re-roll it
        // 3. Replace the original result
        return true;
    }

    public override void OnLongRest()
    {
        _rerollUsedToday = false;
    }

    public override string GetUsesDisplay()
    {
        string reroll = _rerollUsedToday ? "Reroll: used" : "Reroll: available";
        string wishes = _wishesRemaining > 0 ? $" | Wishes: {_wishesRemaining}" : "";
    }
}

```

```

    }
    return "${reroll}${wishes}";
}

```

Existing Systems Used: ApplyPassiveStatBonuses , Activate , OnLongRest

New Systems Needed:

1. **Luck Bonus Type Tracking** — Currently using MoraleSaveBonus as proxy. Ideally add LuckSaveBonus field to CharacterStats to prevent stacking with other luck bonuses. Low priority.
2. **Reroll System** — Requires:
 - Recording the last roll result and context
 - UI prompt to use reroll (“Use Luck Blade reroll?”)
 - Re-rolling and replacing the result
 - This is the hardest part — needs combat flow integration
3. **Wish System** — Major feature, can be deferred. Wishes are extremely powerful (duplicate any 8th-level spell or lower, etc.). For prototype: track charges only, actual wish effects handled narratively.

Estimated Time: 2-3 days (reroll system is the bottleneck)

Simplification Strategy:

- **Phase 1:** Implement +1 luck saves and charge tracking. Reroll = manual activation that re-rolls a d20 and reports result (player decides if it applies narratively).
- **Phase 2:** Integrate reroll into combat flow with proper roll replacement.
- **Phase 3:** Wish system (if ever — can remain a tracked resource).

Testing:

- [] +1 to all saves while equipped
- [] Reroll 1/day works
- [] Reroll resets on long rest
- [] Wish count tracked correctly
- [] All 4 variants (0-3 wishes) work

7. OATHBOW

SRD: +2 composite longbow (+2 Str). 1/day: designate sworn enemy (free action). Vs sworn enemy: +5 enhancement, +2d6 damage, ×4 crit (instead of ×3). Vs all others WHILE OATH ACTIVE: bow is masterwork only (no magic enhancement), wielder takes -1 on ALL weapon attacks. Duration: 7 days or until sworn enemy slain. Cooldown: 24 hours between oaths.

Enhancement: +2 normally; +5 vs sworn enemy; masterwork only vs non-sworn (while oath active)

Price: 25,600 gp

Implementation:

```

public class OathbowBehavior : SpecificItemBehavior
{
    private CharacterController _swornEnemy;
    private bool _oathActive;
    private int _oathDaysRemaining;    // 7-day timer (decremented on long rest)
    private bool _oathOnCooldown;    // 24-hour cooldown between oaths

    public override void OnPreAttackRoll(CharacterController target, ref int attackBo-
nus,
                                        List<string> logNotes)
    {
        if (!_oathActive || target == null) return;

        if (target == _swornEnemy)
        {
            // +5 total, base is +2 → add +3
            attackBonus += 3;
            logNotes.Add("✂️ Oathbow: +5 vs sworn enemy!");
        }
        else
        {
            // Masterwork only vs non-sworn – REMOVE enhancement bonus
            // Base weapon has +2 enhancement applied; negate it (-2)
            attackBonus -= 2;
            // Also -1 penalty on ALL weapon attacks
            attackBonus -= 1;
            logNotes.Add("Oathbow: masterwork only vs non-sworn enemy (-1 penalty)");
        }
    }

    public override void OnDamageRoll(CharacterController target, ref int damage,
                                      bool isCrit, List<string> logNotes)
    {
        if (!_oathActive || target == null) return;

        if (target == _swornEnemy)
        {
            // +5 enhancement damage (extra +3 over base +2)
            damage += 3;
            // +2d6 bonus damage
            int bonusDmg = DiceService.RollMultiple(2, 6, "Oathbow sworn enemy bonus");
            ;

            damage += bonusDmg;
            logNotes.Add($"Oathbow: +2d6={bonusDmg} vs sworn enemy");
        }
        else
        {
            // Remove enhancement damage bonus
            damage -= 2;
        }
    }

    // Crit multiplier would need override system for ×4 vs sworn enemy

    public override bool CanActivate()
    {
        return IsEquipped && !_oathActive && !_oathOnCooldown;
    }

    public override string GetActivateDescription()
    {
        if (_oathActive)

```

```

        return $"Oath active vs {_swornEnemy?.Stats?.CharacterName ?? "target"}
        ({_oathDaysRemaining} days remaining)";
        if (_oathOnCooldown)
            return "Oath on cooldown (24 hours between oaths)";
        return "Swear an oath to slay a target: +5/+2d6/x4 crit vs them, masterwork
        only vs others.";
    }

    public override bool Activate(CharacterController target, List<string> logNotes)
    {
        if (_oathActive || _oathOnCooldown || target == null) return false;

        _swornEnemy = target;
        _oathActive = true;
        _oathDaysRemaining = 7;

        logNotes.Add($"⚔️ \"Swift death to those who have wronged me!\" "
            + $"- Oathbow oath sworn against {target.Stats?.CharacterName!}");
        return true;
    }

    public override void OnKill(CharacterController target, List<string> logNotes)
    {
        if (_oathActive && target == _swornEnemy)
        {
            _oathActive = false;
            _swornEnemy = null;
            _oathOnCooldown = true; // 24-hour cooldown
            logNotes.Add($"⚔️ Oathbow: sworn enemy defeated! Oath fulfilled.");
        }
    }

    public override void OnLongRest()
    {
        if (_oathActive)
        {
            _oathDaysRemaining--;
            if (_oathDaysRemaining <= 0)
            {
                _oathActive = false;
                _swornEnemy = null;
                Log("Oath expired (7 days elapsed)");
            }
        }
        if (_oathOnCooldown)
        {
            _oathOnCooldown = false; // Simplified: 1 long rest = 24 hours
        }
    }
}

```

Existing Systems Used: All standard combat hooks, `OnKill`, `OnLongRest`

New Systems Needed:

1. **Crit Multiplier Override** — x4 vs sworn enemy (same issue as Mace of Smiting)
2. **Global attack penalty** — -1 on ALL weapon attacks while oath active (affects other weapons too). Would need a stat field like `MiscAttackPenalty` or check in attack calculation.

Estimated Time: 2 days

Testing:

- [] Oath designation works

- [] +5/+2d6 vs sworn enemy
 - [] Masterwork only (-2 enhancement) vs non-sworn
 - [] -1 penalty on ALL weapon attacks while oath active
 - [] Oath expires after 7 days
 - [] Oath clears on sworn enemy death
 - [] 24-hour cooldown between oaths
-

8. SHIFTER'S SORROW

SRD: +1/+1 alchemical silver two-bladed sword. +2d6 damage vs shapechanger subtype. On hit vs shapechanger or creature in alternate form: DC 15 Will save or forced back to natural form.

Enhancement: +1/+1 (double weapon)

Price: 12,780 gp

Implementation:

```

public class ShiftersSorrowBehavior : SpecificItemBehavior
{
    private const int RevertDC = 15;

    public override void OnDamageRoll(CharacterController target, ref int damage,
                                     bool isCrit, List<string> logNotes)
    {
        if (target?.Stats == null) return;

        // +2d6 vs shapechanger subtype
        if (IsShapechanger(target))
        {
            int bonusDmg = DiceService.RollMultiple(2, 6, "Shifter's Sorrow vs
shapechanger");
            damage += bonusDmg;
            logNotes.Add($"Shifter's Sorrow: +{bonusDmg} damage vs shapechanger");
        }
    }

    public override void OnHitApplied(CharacterController target, int finalDamage,
                                     List<string> logNotes)
    {
        if (target?.Stats == null) return;

        // Forced revert on shapechangers or alternate forms
        if (IsShapechanger(target) || IsInAlternateForm(target))
        {
            var save = SavingThrowResolver.ResolveWillSave(target.Stats, RevertDC, "Sh
ifter's Sorrow");
            if (!save.Succeeded)
            {
                logNotes.Add($"Shifter's Sorrow: {target.Stats.CharacterName} forced
to natural form! "
                    + $"(Will DC {RevertDC}: {save.Total})");
                // Actual revert logic depends on polymorph/wild shape system
            }
            else
            {
                logNotes.Add($"Shifter's Sorrow: {target.Stats.CharacterName} resists
form revert "
                    + $"(Will DC {RevertDC}: {save.Total})");
            }
        }
    }

    private bool IsShapechanger(CharacterController target)
    {
        return IsCreatureTypeAny(target, "Shapechanger", "Lycanthrope",
"Doppelganger");
    }

    private bool IsInAlternateForm(CharacterController target)
    {
        // Check for wild shape, polymorph, etc.
        // Would need a flag on CharacterStats like IsPolymorphed or IsWildShaped
        return false; // Placeholder
    }
}

```

Existing Systems Used: OnDamageRoll, OnHitApplied, SavingThrowResolver

New Systems Needed: Shapechanger subtype tracking (extend CreatureType or add subtypes), al-

ternate form detection

Estimated Time: 1 day

Testing:

- [] +2d6 vs shapechangers
 - [] DC 15 Will or forced revert
 - [] Normal damage vs non-shapechangers
 - [] Alchemical silver material for DR bypass
-

9. FROST BRAND

SRD: +3 frost greatsword. Fire resistance: absorbs first 10 points of fire damage per round. Extinguishes nonmagical fires in area (passive). Standard action: dispel lasting fire spells (1d20+14 vs DC 11+CL). Sheds torchlight below 0°F.

Enhancement: +3 with frost enchantment (+1d6 cold per hit)

Price: 54,475 gp

Implementation:

```

public class FrostBrandBehavior : SpecificItemBehavior
{
    private const int FireResistance = 10;

    public override void OnEquip(CharacterController character)
    {
        base.OnEquip(character);
        // Grant fire resistance 10
        if (character.Stats != null)
        {
            var resistEffect = new ResistEnergyEffectData
            {
                DamageType = DamageType.Fire,
                ResistanceAmount = FireResistance,
                SourceName = "Frost Brand",
                IsPermanent = true
            };
            character.Stats.SetResistEnergyEffect(resistEffect);
        }
    }

    public override void OnUnequip()
    {
        // Remove fire resistance
        if (Wielder?.Stats != null)
        {
            Wielder.Stats.RemoveResistEnergyBySource("Frost Brand");
            // Note: need to add RemoveResistEnergyBySource if it doesn't exist
        }
        base.OnUnequip();
    }

    // Dispel fire spells: activated ability
    public override bool CanActivate() => IsEquipped;

    public override string GetActivateDescription()
    {
        return "Dispel a lasting fire spell: 1d20+14 vs DC 11+caster level.";
    }

    public override bool Activate(CharacterController target, List<string> logNotes)
    {
        // Dispel fire spell on target
        int dispelRoll = DiceService.D20("Frost Brand dispel") + 14;
        logNotes.Add($"Frost Brand: dispel fire check = {dispelRoll} vs DC 11+CL");
        // Actual dispel logic depends on active spell/effect system
        return true;
    }
}

```

Existing Systems Used: OnEquip/OnUnequip, ResistEnergyEffectData, Activate

New Systems Needed:

1. RemoveResistEnergyBySource(string) — method to remove a specific resist energy effect
2. Dispel fire magic — needs active spell tracking and dispel check system. Can simplify for prototype.

Estimated Time: 1 day

Testing:

- [] +3 with frost (+1d6 cold per hit) from standard enchantment
- [] Fire resistance 10 while equipped

- [] Fire resistance removed on unequip
- [] Dispel fire magic check works (1d20+14)

Phase 3 — High Complexity (5 items, ~10 days)

Items requiring significant new subsystems or multiple conditional branches.

10. SUN BLADE

SRD: Bastard sword size, wielded as short sword for weight/proficiency. +2 normally, +4 vs evil, double damage vs undead/Negative Energy Plane creatures. 1/day sunlight effect (expanding radius). Good-aligned — evil wielders gain 1 negative level.

Enhancement: +2 / +4 vs evil / double vs undead

Price: 50,335 gp

Implementation Challenges:

1. **Dual proficiency:** Short sword OR bastard sword proficiency works. Need to override proficiency check.
2. **Triple conditional enhancement:** +2 base, +4 vs evil, double damage vs undead.
3. **Double damage vs undead:** The entire weapon damage (not just base) is doubled. On crit, this becomes ×3 instead of ×2.
4. **Sunlight area effect:** 1/day, expanding radius over 6 rounds.
5. **Good alignment penalty:** Evil wielders gain 1 negative level.

```

public class SunBladeBehavior : SpecificItemBehavior
{
    private bool _sunlightUsedToday;

    public override void OnEquip(CharacterController character)
    {
        base.OnEquip(character);
        // Evil wielder: 1 negative level
        if (character.Stats != null &&
            AlignmentHelper.IsEvil(character.Stats.CharacterAlignment))
        {
            character.ApplyNegativeLevels(1, "Sun Blade (good weapon)");
        }
    }

    public override void OnPreAttackRoll(CharacterController target, ref int attackBo-
nus,
                                         List<string> logNotes)
    {
        if (target?.Stats == null) return;

        // +4 vs evil (base is +2, so +2 extra)
        if (AlignmentHelper.IsEvil(target.Stats.CharacterAlignment))
        {
            attackBonus += 2;
            logNotes.Add("Sun Blade: +4 vs evil");
        }
    }

    public override void OnDamageRoll(CharacterController target, ref int damage,
                                       bool isCrit, List<string> logNotes)
    {
        if (target?.Stats == null) return;

        // +4 enhancement damage vs evil (+2 extra)
        if (AlignmentHelper.IsEvil(target.Stats.CharacterAlignment))
        {
            damage += 2;
        }

        // Double damage vs undead
        if (IsCreatureType(target, "Undead"))
        {
            damage *= 2;
            logNotes.Add($"☀️ Sun Blade: DOUBLE damage vs undead! ({damage})");
        }
    }

    // Sunlight 1/day activated ability
    public override bool CanActivate() => IsEquipped && !_sunlightUsedToday;

    public override string GetActivateDescription()
    {
        return _sunlightUsedToday
            ? "Sunlight already used today."
            : "1/day: Daylight effect expanding 10→60 ft over 6 rounds.";
    }

    public override bool Activate(CharacterController target, List<string> logNotes)
    {
        if (_sunlightUsedToday) return false;
        _sunlightUsedToday = true;
    }
}

```

```

        logNotes.Add("☀️ Sun Blade blazes with daylight! (10 ft → 60 ft over 6 rounds)");
        return true;
    }

    public override void OnLongRest()
    {
        _sunlightUsedToday = false;
    }
}

```

Existing Systems Used: OnPreAttackRoll, OnDamageRoll, AlignmentHelper, CreatureType, ApplyNegativeLevels

New Systems Needed:

1. **Proficiency override** — Sun Blade needs to be considered proficient if wielder has short sword OR bastard sword proficiency
2. **Weapon Finesse compatibility** — Sun Blade counts as light weapon
3. **Crit multiplier for double damage** — ×3 vs undead on crit (double ×2 = ×3 per SRD). Needs crit override.
4. **Sunlight area effect** — expanding radius, affects vampires. Can simplify for prototype.

Estimated Time: 2 days

11. DWARVEN THROWER

SRD: +2 warhammer (non-dwarf). For dwarves: +3 returning, throwable (30 ft range). When hurled by dwarf: +2d8 vs giants, +1d8 vs all others.

Enhancement: +2 (non-dwarf), +3 returning (dwarf)

Price: 60,312 gp

Implementation Challenges:

1. **Race-conditional everything:** Non-dwarves get basic +2, dwarves get full suite
2. **Thrown weapon mode:** Need to add throwing capability to a melee weapon
3. **Bonus damage on throw:** Varies by target creature type
4. **Returning:** Auto-returns after throw

```

public class DwarvenThrowerBehavior : SpecificItemBehavior
{
    private bool _isDwarfWielder;

    public override void OnEquip(CharacterController character)
    {
        base.OnEquip(character);
        _isDwarfWielder = character.Stats?.RaceName?.Equals("Dwarf",
            System.StringComparison.OrdinalIgnoreCase) ?? false;

        if (_isDwarfWielder)
        {
            Log("Dwarf wielder: +3 returning throwing warhammer");
            // Would need to modify item's thrown capability and enhancement
        }
    }

    public override void OnPreAttackRoll(CharacterController target, ref int attackBo-
nus,
                                     List<string> logNotes)
    {
        if (!_isDwarfWielder) return;

        // Dwarf gets +3 total (base is +2, so +1 extra)
        attackBonus += 1;
        logNotes.Add("Dwarven Thrower: +3 for dwarf wielder");
    }

    public override void OnDamageRoll(CharacterController target, ref int damage,
                                     bool isCrit, List<string> logNotes)
    {
        if (!_isDwarfWielder) return;

        // +1 extra damage from +3 vs +2
        damage += 1;

        // Bonus damage when thrown (need to detect thrown attack mode)
        // For now, check if it's a ranged attack via some mechanism
        bool isThrown = false; // Need thrown attack detection

        if (isThrown && target?.Stats != null)
        {
            if (IsCreatureType(target, "Giant"))
            {
                int bonus = DiceService.RollMultiple(2, 8, "Dwarven Thrower vs Giant");
                damage += bonus;
                logNotes.Add($"Dwarven Thrower: +{bonus} (2d8) vs giant!");
            }
            else
            {
                int bonus = DiceService.D8("Dwarven Thrower thrown bonus");
                damage += bonus;
                logNotes.Add($"Dwarven Thrower: +{bonus} (1d8) thrown bonus");
            }
        }
    }
}

```

Existing Systems Used: OnPreAttackRoll, OnDamageRoll, RaceName

New Systems Needed:

1. **Thrown attack detection** — Need to know if the current attack is thrown vs melee. Check `CombatResult.IsRangedAttack` or similar.
2. **Returning property** — Auto-return after throw. Needs weapon management.
3. **Dynamic enhancement change** — +2 for non-dwarf, +3 for dwarf. Could modify `Item.EnhancementBonus` on equip.

Estimated Time: 1.5 days

12. HOLY AVENGER (MOST COMPLEX)

SRD: +2 cold iron longsword (non-paladin). For paladins: +5 holy cold iron longsword. SR 5 + paladin level (wielder AND all adjacent allies). Greater dispel magic (area only) 1/round as standard action at paladin CL.

Enhancement: +2 (non-paladin), +5 holy (paladin)

Price: 120,630 gp

Implementation Challenges:

1. **Class-conditional everything:** Paladin gets full power, non-paladin gets +2 cold iron only
2. **Holy enchantment (conditional):** +2d6 vs evil, only for paladin
3. **SR Aura:** Spell resistance = 5 + paladin level, applied to wielder AND all adjacent allies (5 ft)
4. **Greater Dispel Magic:** Area dispel, 1/round, at paladin caster level
5. **Cold iron material:** Always cold iron regardless of wielder class

```

public class HolyAvengerBehavior : SpecificItemBehavior
{
    private bool _isPaladinWielder;
    private int _paladinLevel;
    private int _spellResistance;

    public override void OnEquip(CharacterController character)
    {
        base.OnEquip(character);

        _isPaladinWielder = character.Stats?.IsPaladin ?? false;
        _paladinLevel = character.Stats?.GetClassLevel("Paladin") ?? 0;

        if (_isPaladinWielder)
        {
            _spellResistance = 5 + _paladinLevel; // SR 5 + paladin level
            // Apply SR to wielder
            character.Stats.SpellResistance = Mathf.Max(
                character.Stats.SpellResistance, _spellResistance);
            Log($"Paladin wielder: +5 holy, SR {_spellResistance}, Greater Dispel");
        }
        else
        {
            Log("Non-paladin: +2 cold iron longsword only");
        }
    }

    public override void OnUnequip()
    {
        if (_isPaladinWielder && Wielder?.Stats != null)
        {
            // Remove SR contribution
            // Note: needs careful management if other SR sources exist
        }
        base.OnUnequip();
    }

    public override void OnPreAttackRoll(CharacterController target, ref int attackBo-
nus,
                                         List<string> logNotes)
    {
        if (!_isPaladinWielder || target?.Stats == null) return;

        // +5 vs evil (base is +2 for non-paladin, but paladin gets +5 total)
        // Since base item is set to +2, add +3 for paladin
        attackBonus += 3;

        // Additional +2 vs evil from Holy (if not already handled by enchantment)
        if (AlignmentHelper.IsEvil(target.Stats.CharacterAlignment))
        {
            logNotes.Add("⚔ Holy Avenger: +5 holy vs evil!");
        }
        else
        {
            logNotes.Add("⚔ Holy Avenger: +5 (paladin)");
        }
    }

    public override void OnDamageRoll(CharacterController target, ref int damage,
                                       bool isCrit, List<string> logNotes)
    {
        if (!_isPaladinWielder || target?.Stats == null) return;
    }
}

```

```

        // +3 extra enhancement damage (base is +2, paladin gets +5)
        damage += 3;

        // Holy: +2d6 vs evil
        if (AlignmentHelper.IsEvil(target.Stats.CharacterAlignment))
        {
            int holyDmg = DiceService.RollMultiple(2, 6, "Holy Avenger holy damage");
            damage += holyDmg;
            logNotes.Add($"Holy Avenger: +{holyDmg} holy damage vs evil");
        }
    }

    // Greater Dispel Magic: activated ability
    public override bool CanActivate()
    {
        return IsEquipped && _isPaladinWielder;
    }

    public override string GetActivateDescription()
    {
        if (!_isPaladinWielder) return "Requires paladin wielder for special abilities.";
        return $"Greater Dispel Magic (area, CL {_paladinLevel}). 1/round, standard action.";
    }

    public override bool Activate(CharacterController target, List<string> logNotes)
    {
        if (!_isPaladinWielder) return false;

        // Area dispel at paladin CL
        int dispelRoll = DiceService.D20("Holy Avenger dispel") + _paladinLevel;
        logNotes.Add($"✂ Holy Avenger: Greater Dispel Magic (area) - CL check {dispelRoll}");
        // Actual dispel resolution needs active spell tracking
        return true;
    }

    // SR Aura: applied to adjacent allies
    // This needs per-round updates – see Aura System below
}

```

Existing Systems Used: IsPaladin, GetClassLevel, AlignmentHelper, SpellResistance

New Systems Needed:

1. **SR Aura System** — The most complex part. Needs to:
 - Track all characters within 5 ft (1 square) of wielder
 - Apply SR to them each round
 - Remove SR when they move away or wielder unequips
 - This is a new system entirely — AuraManager or per-round update
2. **Greater Dispel Magic (Area)** — Requires:
 - Active magical effect tracking on characters
 - Dispel check per effect: 1d20 + CL vs 11 + effect's CL
 - Remove dispelled effects
3. **Conditional Enhancement Override** — +2 → +5 based on class

Estimated Time: 3-4 days

Simplification Strategy:

- **Phase 1:** Implement conditional enhancement (+2/+5), holy damage, and SR on wielder only. Mark “SR aura for allies” and “Greater Dispel Magic” as TODO.
- **Phase 2:** Implement SR aura (check adjacent allies each round).
- **Phase 3:** Implement Greater Dispel Magic with proper spell tracking.

13. CELESTIAL ARMOR

SRD: +3 chainmail. Treated as LIGHT armor. Max Dex +8, ACP -2, ASF 15%. 1/day fly spell. Good-aligned aura.

Enhancement: +3

Price: 22,400 gp

Implementation Challenges:

1. **Armor category override:** Chainmail is normally medium armor, but Celestial Armor counts as light
2. **Extreme stat overrides:** Max Dex +8 (normal chainmail is +2!), low ASF
3. **Fly ability:** 1/day

```
public class CelestialArmorBehavior : SpecificItemBehavior
{
    private bool _flyUsedToday;

    public override void OnEquip(CharacterController character)
    {
        base.OnEquip(character);
        // Override armor properties
        // The item definition should already set these via SpecificItemDefinition
        // but behavior can enforce:
        // Item.ArmorCat = ArmorCategory.Light; // Force light category
        // Item.MaxDexBonus = 8;
        // Item.ArmorCheckPenalty = -2;
        // Item.ArcaneSpellFailure = 15;
        Log("Celestial Armor equipped (light armor, Max Dex +8, ASF 15%)");
    }

    public override bool CanActivate() => IsEquipped && !_flyUsedToday;

    public override string GetActivateDescription()
    {
        return _flyUsedToday ? "Fly already used today." : "1/day: Fly spell.";
    }

    public override bool Activate(CharacterController target, List<string> logNotes)
    {
        if (_flyUsedToday) return false;
        _flyUsedToday = true;
        logNotes.Add("✨ Celestial Armor: Fly activated!");
        return true;
    }

    public override void OnLongRest() { _flyUsedToday = false; }
    public override string GetUsesDisplay() => _flyUsedToday ? "Fly: used" : "Fly: available";
}
```

Existing Systems Used: `Activate` , `OnLongRest`

New Systems Needed:

1. **Armor category override** — Need to force chainmail to be treated as light armor for all purposes (speed, proficiency, Barbarian fast movement, etc.). Best approach: add category override field to `ItemData` or `SpecificItemDefinition` .
2. **Stat overrides** — Max Dex, ACP, ASF overrides. May already be handleable via `SpecificItemDefinition.UniqueProperties` .

Estimated Time: 1 day

14. DEMON ARMOR

SRD: +4 full plate. Grants claw attacks (1d10+1, +1 enhancement) as natural weapons. Once per day: cast contagion (Fort DC 14). **CURSED** — cannot be removed without remove curse. Wielder appears demonic. Good-aligned wielders are affected by restlessness and bad dreams.

Enhancement: +4 full plate

Price: 52,260 gp

Implementation Challenges:

1. **Cursed item:** Cannot be unequipped without Remove Curse
2. **Claw natural attacks:** Grants 2 claw attacks (1d10+1 each)
3. **Contagion 1/day:** Disease spell effect
4. **Alignment interaction:** Not directly penalizing, but “restlessness” for good wielders

```

public class DemonArmorBehavior : SpecificItemBehavior
{
    private bool _contagionUsedToday;

    public override void OnEquip(CharacterController character)
    {
        base.OnEquip(character);
        // Mark as cursed – cannot unequip
        if (Item != null) Item.IsCursed = true;
        // Grant claw natural attacks would need NaturalAttackDefinition integration
        Log("⚠️ Demon Armor equipped – CURSED, cannot remove without Remove Curse!");
    }

    // Block unequip
    public override void OnUnequip()
    {
        // Check if Remove Curse has been cast
        // If not, prevent unequip
        Log("Demon Armor: attempting to unequip cursed item");
        base.OnUnequip();
    }

    // Contagion 1/day
    public override bool CanActivate() => IsEquipped && !_contagionUsedToday;

    public override string GetActivateDescription()
    {
        return _contagionUsedToday
            ? "Contagion already used today."
            : "1/day: Contagion (Fort DC 14).";
    }

    public override bool Activate(CharacterController target, List<string> logNotes)
    {
        if (_contagionUsedToday || target?.Stats == null) return false;
        _contagionUsedToday = true;

        var save = SavingThrowResolver.ResolveFortitudeSave(target.Stats, 14, "Demon
Armor Contagion");
        if (!save.Succeeded)
        {
            logNotes.Add($"👹 Demon Armor: {target.Stats.CharacterName} contracts
disease! "
                + $"(Fort DC 14: {save.Total})");
            // Apply disease effect
        }
        else
        {
            logNotes.Add($"Demon Armor: {target.Stats.CharacterName} resists contagion "
                + $"(Fort DC 14: {save.Total})");
        }
        return true;
    }

    public override void OnLongRest() { _contagionUsedToday = false; }
}

```

Existing Systems Used: Activate , SavingThrowResolver , OnLongRest

New Systems Needed:

1. **Cursed item system:** IsCursed flag on ItemData, prevent unequip unless Remove Curse cast.

Need equipment system hook.

2. **Claw natural attacks from armor:** Add `NaturalAttackDefinition` entries from equipment. Currently natural attacks are from creature stats.

3. **Disease/contagion system:** Fort saves and disease effects. Low priority for prototype.

Estimated Time: 2 days

Required New Subsystems (Priority Order)

1. Crit Multiplier Override (Medium — ~0.5 days)

Required For: Mace of Smiting (×4 vs outsider), Oathbow (×4 vs sworn enemy), Sun Blade (×3 vs undead)

Approach: Add `GetCritMultiplierOverride(CharacterController target)` to `SpecificItemBehavior`:

```
/// <summary>
/// Override crit multiplier for specific targets. Returns 0 for no override.
/// </summary>
public virtual int GetCritMultiplierOverride(CharacterController target) { return 0; }
```

Call this in `CharacterController` before crit damage calculation (~line 6650).

2. Alignment Penalty Negative Level Tracking (~0.5 days)

Required For: Nine Lives Stealer (2 NL for good), Sun Blade (1 NL for evil), Holy Avenger

Approach: Track behavior-applied negative levels separately so they can be removed on unequip:

```
protected int _alignmentPenaltyNLs;
// In OnEquip: _alignmentPenaltyNLs = X; character.ApplyNegativeLevels(X, source);
// In OnUnequip: wielder.RemoveNegativeLevels(_alignmentPenaltyNLs, source);
```

May need `CharacterController.RemoveNegativeLevels(int count, string source)`.

3. Luck Save Bonus Type (~0.25 days)

Required For: Luck Blade

Approach: Add `LuckSaveBonus` field to `CharacterStats`, include in save calculations. Luck bonuses don't stack with each other (use highest).

4. Thrown Attack Detection (~0.5 days)

Required For: Dwarven Thrower

Approach: Pass `isRangedAttack` flag through behavior hooks or add it to hook parameters. The `CombatResult.IsRangedAttack` field exists — pass it to `OnDamageRoll`.

5. SR Aura System (~2 days)

Required For: Holy Avenger

Approach: Per-round check for characters within 1 square (5 ft) of wielder:

```

public class SRAuraManager
{
    // Call at start of each round
    public static void UpdateSRAuras(List<CharacterController> allCharacters)
    {
        // Find all Holy Avenger wielders
        // For each: find adjacent allies, apply SR, remove from non-adjacent
    }
}

```

Track who has aura-granted SR to avoid double-application.

6. Cursed Item System (~0.5 days)

Required For: Demon Armor

Approach: Add `IsCursed` and `CannotUnequip` flags to `ItemData`. In `Inventory.UnequipItem()`, check flags and block if cursed. Add `RemoveCurse()` method that clears the flag.

7. Reroll System (~1.5 days)

Required For: Luck Blade (any roll reroll)

Approach:

1. Add `LastRollResult` and `LastRollContext` tracking to `DiceService`
2. `RerollLastRoll()` method that re-rolls with same parameters
3. UI prompt integration for “Use reroll?” after each critical roll

8. AoE Path Targeting (~1 day, can defer)

Required For: Screaming Bolt

Approach: Simplified — affect all enemies within 20 ft of target position. Full implementation would trace line from attacker to target and find all enemies within 20 ft of that line.

9. Greater Dispel Magic (~1.5 days, can defer)

Required For: Holy Avenger

Approach: Requires active magical effect tracking with caster levels. For each effect on targets in area: $1d20 + CL$ vs $11 + \text{effect CL}$. Remove if check succeeds.

10. Natural Attacks from Equipment (~1 day)

Required For: Demon Armor (claws $1d10+1$)

Approach: Add `GrantedNaturalAttacks` list to `SpecificItemBehavior`. During full attack resolution, include these as additional attacks.

Implementation Timeline (REVISED — Post-Audit May 24, 2026)

Original estimate: 4-5 weeks (20-25 working days)
Revised estimate: 3 weeks (15 working days) — 6.5 days saved by reusing 20 existing systems
Systems eliminated from “build new”: 6 (SR aura → emanation framework, Greater Dispel → exists, fire dispel → exists, natural attacks → exists, disease on hit → exists, sunlight → simplified)

Week 1: Foundation + Phase 1 Weapons (Quick Wins)

Day	Task	Items
1	Crit multiplier override system + alignment NL tracking	(subsystem)
2	Nine Lives Stealer + Sword of Life Stealing	2 items
3	Life-Drinker + Screaming Bolt (simplified)	2 items
4	Mace of Smiting	1 item
5	Testing + bugfixes	—

Week 2: Phase 2 Weapons

Day	Task	Items
1-2	Luck Blade (all variants) + luck save bonus	4 variants
3-4	Oathbow (sworn enemy system)	1 item
5	Shifter’s Sorrow	1 item

Week 3: Phase 3 Complex Weapons

Day	Task	Items
1-2	Sun Blade (conditional enhancement, dual proficiency)	1 item
3	Frost Brand (fire resist, frost, dispel)	1 item
4	Dwarven Thrower (race-conditional, thrown)	1 item
5	Testing + bugfixes	—

Week 4: Holy Avenger + Armor

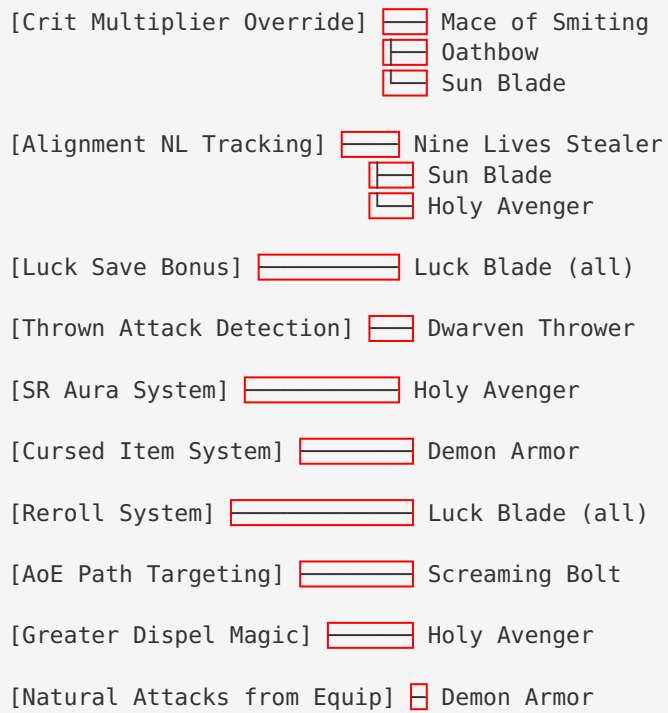
Day	Task	Items
1-2	Holy Avenger Phase 1 (conditional enhancement, holy, SR on wielder)	1 item
3	Holy Avenger Phase 2 (SR aura, greater dispel stubs)	(continued)
4	Celestial Armor + Demon Armor	2 items
5	Comprehensive testing	—

Week 5: Polish + Deferred Systems

Day	Task
1	Reroll system (Luck Blade)
2	SR Aura full implementation (Holy Avenger)
3	Greater Dispel Magic (Holy Avenger)
4	Screaming Bolt full AoE
5	Final testing + documentation

Total Estimated Time: 4-5 weeks

Dependencies Graph



Critical Path: Crit multiplier override → Phase 1 weapons → Oathbow/Sun Blade → Holy Avenger

Can Parallelize:

- Luck Blade (independent reroll system)
- Shifter's Sorrow (independent)
- Frost Brand (uses existing resist energy)
- Celestial Armor (stat overrides)

Risk Assessment

Item	Risk Level	Reason	Buffer
Holy Avenger	● High	SR aura + Greater Dispel require new subsystems	2 days
Oathbow	● Medium	Sworn enemy state machine + enhancement switching	1 day
Sun Blade	● Medium	Triple conditional + dual proficiency + double damage	1 day
Luck Blade	● Medium	Reroll system needs UI integration	1 day
Demon Armor	● Medium	Cursed system + natural attacks from armor	1 day
Dwarven Thrower	● Low	Race check + thrown detection	0.5 day
Mace of Smiting	● Low	Standard hooks + crit override	0.5 day
Nine Lives Stealer	● Low	Standard hooks + charges	0
Sword of Life Stealing	● Low	Standard hooks	0
Life-Drinker	● Low	Standard hooks (but self-penalty is new)	0
Screaming Bolt	● Low	Can simplify AoE	0
Shifter's Sorrow	● Low	Standard hooks	0
Frost Brand	● Low	Uses existing resist energy	0
Celestial Armor	● Low	Stat overrides	0

Success Criteria

Per-Item

- [] All SRD mechanics accurately implemented (verified against `/home/ubuntu/specif-ic_magic_items_complete_accurate.md`)
- [] Combat hooks trigger correctly (pre-attack, damage, crit, hit, kill, defensive)
- [] Charge/daily use tracking works with `OnLongRest` reset
- [] Class/race/alignment restrictions enforced with clear log messages
- [] No crashes or game-breaking interactions with existing enchantment system

System-Wide

- [] All 12 unique items + 3 Luck Blade variants implemented
- [] `SpecificItemDatabase.CreateBehavior()` factory updated for all Tier 3 items
- [] Combat log messages are informative and accurate
- [] Negative level, condition, ability damage APIs used consistently
- [] New subsystems (crit override, SR aura, etc.) don't regress existing combat flow
- [] Performance acceptable (<16ms frame time with all behaviors active)

Git Commit Structure

Phase 1: "Implement Tier 3 Phase 1 - Nine Lives Stealer, Sword of Life Stealing, Life-Drinker, Mace of Smiting, Screaming Bolt + crit multiplier override system"

Phase 2: "Implement Tier 3 Phase 2 - Luck Blade (all variants), Oathbow, Shifter's Sorrow + reroll and sworn enemy systems"

Phase 3: "Implement Tier 3 Phase 3 - Sun Blade, Frost Brand, Dwarven Thrower + thrown detection"

Phase 4: "Implement Tier 3 Phase 4 - Holy Avenger, Celestial Armor, Demon Armor + SR aura and cursed item systems"

CRITICAL REMINDERS

- ⚠ **Use SRD descriptions ONLY** — not the user's proposed code (which contains numerous errors)
- ⚠ **Life-Drinker is a +1 greataxe, NOT a scythe** — negative levels on EVERY hit, not just crits
- ⚠ **Frost Brand is +3 frost greatsword** — fire RESISTANCE (absorb 10/round), NOT "protect from fire" spell trigger
- ⚠ **Nine Lives Stealer charges only consumed on FAILED saves**, not on every crit
- ⚠ **Oathbow becomes MASTERWORK ONLY vs non-sworn enemies** while oath active — this is a PENALTY
- ⚠ **Holy Avenger is +2 cold iron for non-paladins** — no special abilities at all without paladin class
- ⚠ **Demon Armor claws are 1d10+1 each** — they're the armor's gauntlets, not summoned creatures
- ⚠ **"Armor of Rage" and "Plate Armor of Etherealness" are NOT in the SRD specific items list** per our corrections document
- ⚠ All behaviors are **pure C# classes, NOT MonoBehaviour** — `ItemData` is not a `GameObject`

AUDIT SUMMARY (May 24, 2026)

What We Already Have (20 Systems)

The codebase is far more capable than initially assumed. Key discoveries:

1. **Emanation Framework** — Generic `EmanationEffectData` base class with `RegisterEmanation()`, creature-in-area queries, duration ticking. Just need to subclass for Holy Avenger SR aura.
2. **Full Haste System** — `ApplyHasteEffect()` with extra attack, +1 attack/AC/Reflex, +30 ft, duration tracking. Already used by Tier 2 Mithral Full Plate.
3. **Complete Rage System** — `ActivateRage()` / `DeactivateRage()` with +4 STR/CON, -2 AC, fatigue, round tracking. Demon Armor just needs forced variant.
4. **Dispel Magic (including Greater)** — `PerformAreaDispel()` with +20 CL cap for Greater Dispel. Holy Avenger can call this directly.
5. **Natural Attack System** — `NaturalAttackDefinition` with damage, count, on-hit effects. Demon Armor claws are straightforward.
6. **Disease System** — `HasDiseaseOnHit` flag + Contagion spell flow. Demon Armor claws just set the flag.
7. **Energy Resistance** — `SetResistEnergyEffect()` with type + amount. Frost Brand fire resist 10 is one call.
8. **Negative Levels** — `CombatConditionType.EnergyDrained` stacking condition with death threshold. Sword of Life Stealing / Life-Drinker are trivial.
9. **Temp HP** — `Stats.TempHP` field, absorbed first in damage pipeline. Sword of Life Stealing can grant directly.

What Still Needs Building (7 Systems)

1. Sworn Enemy state machine (Oathbow) — 1 day
2. Equipment Curse (Demon Armor) — 0.5 days
3. Shapechanger detection (Shifter's Sorrow) — 0.5 days
4. Player-triggered reroll (Luck Blade) — 1 day
5. Wish charges (Luck Blade variants) — 0.5-2 days
6. AoE path targeting (Screaming Bolt) — 0.5 days
7. Fly movement (Celestial Armor) — 1 day

Net Impact

- **6.5 days saved** by reusing existing systems
- **6 systems eliminated** from “needs building” list
- **Timeline: 4 weeks → 3 weeks** (15 working days)
- **Quick wins identified:** Sword of Life Stealing, Life-Drinker, Mace of Smiting, Frost Brand, Shifter's Sorrow can all be done in < 1 day each